

BUỔI 7 - Form CRUD Môn học & Đồng bộ Trạng thái

Điều kiện tiên quyết: đã hoàn thành Buổi 6 - CourseList, SearchBox, Pagination, useCourses hoạt động đúng, xử lý đủ 4 trạng thái Loading/Success/Empty/Error.

Buổi này thêm mới khả năng Thêm/Sửa/Xoá vào đúng màn hình đã có, tái sử dụng nguyên vẹn CourseList và useCourses.

0. Mục tiêu buổi học

Sau buổi học này, sinh viên phải:

- **1. Viết CourseForm** dạng Controlled Component dùng chung cho cả Thêm và Sửa - không viết 2 form riêng biệt.
- **2. Validate** dữ liệu phía Client trước khi gửi lên server (bổ sung cho Validation phía Server đã có từ Buổi 2, không thay thế).
- **3. Gọi POST/PUT /api/courses** qua Gateway - lần đầu tiên Frontend gửi kèm JWT trong Header, vì các endpoint này yêu cầu ROLE_ADMIN (đã cấu hình ở Buổi 4).
- **4. Hiển thị đúng lỗi trả về** từ course-service (đi qua Gateway) lên giao diện, tái sử dụng ApiErrorResponse đã viết ở Buổi 5.
- **5. Đồng bộ lại danh sách** (refetch) sau khi Thêm/Sửa/Xoá thành công, không cần load lại cả trang.

A. Vấn đề cần giải quyết trước: chưa có màn hình Đăng nhập

POST/PUT/DELETE /courses yêu cầu ROLE_ADMIN (cấu hình từ Buổi 4), nghĩa là request phải có Header Authorization: Bearer <token>. Tuy nhiên màn hình Đăng nhập thật (/login, AuthContext, lưu token tự động) chỉ được xây ở Buổi 8.

Giải pháp tạm thời cho buổi này (sẽ được thay thế hoàn toàn, không phải viết lại code khác):

- Dùng Postman gọi POST http://localhost:8080/api/auth/login với tài khoản admin/admin123 (đã seed từ Buổi 4), copy giá trị token trả về.
- Mở trình duyệt tại trang Frontend, mở DevTools Console, chạy lệnh:

```
localStorage.setItem('crs_token', 'DAN_TOKEN_VUA_COPY_VAO_DAY');
```

- axiosClient sẽ được cập nhật (mục B dưới đây) để tự đọc token từ đúng key localStorage này.

Giải thích: đây không phải là làm tắt hay làm sai - key localStorage.crs_token được đặt tên thống nhất ngay từ bây giờ chính là để Buổi sau chỉ cần viết AuthContext ghi vào đúng key này sau khi đăng nhập thật, axiosClient và CourseForm viết ở buổi này không cần sửa lại bất kỳ dòng nào.

Lưu ý: Mặc định thì DevTools Console không cho phép dán (Ctrl + V – Paste) nội dung vào. Để có thể dán:

- Ngay vị trí có dấu nhắc (**>**) gõ trực tiếp cụm từ: **allow pasting** (không cần dấu ngoặc kép) rồi ấn **Enter** để cho phép lệnh dán nội dung

B. Cập nhật axiosClient — tự động đính kèm token (bản tối giản)

```
// path: crs-frontend/src/api/axiosClient.ts
// purpose: axios instance duy nhất của frontend, tro ve api-gateway
// và tu dong dinh kem
// Authorization header neu co token trong localStorage.
// LƯU Ý: đây là bản tối giản - Buổi 8 sẽ bổ sung Response Interceptor
// để tu dong
// đang xuất khi gặp lỗi 401 (token hết hạn), không cần sửa lại đoạn
// code dưới đây.

import axios from 'axios';

const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

axiosClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('crs_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default axiosClient;
```

C. CourseForm - Controlled Component dùng chung Thêm/Sửa

C.1. Nguyên tắc Controlled Component

Mỗi input trong form được điều khiển hoàn toàn bởi React state (value + onChange), không dùng ref để đọc DOM trực tiếp. Điều này cho phép validate ngay khi người dùng gõ, và dễ dàng "mời" sẵn dữ liệu cũ khi ở chế độ Sửa.

C.2. Kiểu dữ liệu cho form

```
// path: crs-frontend/src/types/course.ts
// purpose: bổ sung kiểu dữ liệu riêng cho form, khác với Course
// (Course có id, form thì không bắt buộc)

// (thêm vào cuối file course.ts đã có từ Buổi 5)

export interface CourseFormValues {
  tenMonHoc: string;
  soTinChi: string;    // dùng string trong form để kiểm soát input
  // rong, sẽ parseInt khi gửi đi
  soChoToiDa: string;
}

export const emptyCourseForm: CourseFormValues = {
  tenMonHoc: '',
  soTinChi: '',
  soChoToiDa: '',
};
```

C.3. Component CourseForm

```
// path: crs-frontend/src/components/CourseForm.tsx
// purpose: form dùng chung cho Thêm và Sửa môn học, validate phía
// client trước khi gọi API

import { useState, useEffect } from 'react';
import type { Course, CourseFormValues } from '../types/course';
import { emptyCourseForm } from '../types/course';

interface CourseFormProps {
```

```

    editingCourse: Course | null; // null = dang o che do Them; co gia
    tri = dang Sua
    onSubmit: (values: CourseFormValues) => Promise<void>;
    onCancel: () => void;
    submitting: boolean;
    serverError: string | null;
}

export default function CourseForm({
    editingCourse,
    onSubmit,
    onCancel,
    submitting,
    serverError,
}: CourseFormProps) {
    const [values, setValues] =
    useState<CourseFormValues>(emptyCourseForm);
    const [clientErrors, setClientErrors] =
    useState<Partial<CourseFormValues>>({});

    // Moi lan editingCourse thay doi (bam nut Sua tren 1 dong khac), mo
    lai du lieu vao form
    useEffect(() => {
        if (editingCourse) {
            setValues({
                tenMonHoc: editingCourse.tenMonHoc,
                soTinChi: String(editingCourse.soTinChi),
                soChoToiDa: String(editingCourse.soChoToiDa),
            });
        } else {
            setValues(emptyCourseForm);
        }
        setClientErrors({});
    }, [editingCourse]);

    const validate = (): boolean => {
        const errors: Partial<CourseFormValues> = {};

        if (!values.tenMonHoc.trim()) {
            errors.tenMonHoc = 'Ten mon hoc khong duoc de trong';

```

```

    }

    const soTinChi = Number(values.soTinChi);
    if (!values.soTinChi || isNaN(soTinChi) || soTinChi <= 0) {
        errors.soTinChi = 'So tin chi phai la so lon hon 0';
    }

    const soChoToiDa = Number(values.soChoToiDa);
    if (!values.soChoToiDa || isNaN(soChoToiDa) || soChoToiDa <= 0) {
        errors.soChoToiDa = 'So cho toi da phai la so lon hon 0';
    }

    setClientErrors(errors);
    return Object.keys(errors).length === 0;
};

const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!validate()) return;
    await onSubmit(values);
};

return (
    <form onSubmit={handleSubmit} style={{ border: '1px solid #ddd',
padding: 16, borderRadius: 8, marginBottom: 16 }}>
        <h3>{editingCourse ? 'Sua mon hoc' : 'Them mon hoc moi'}</h3>

        <div style={{ marginBottom: 8 }}>
            <label>Ten mon hoc</label><br />
            <input
                type="text"
                value={values.tenMonHoc}
                onChange={(e) => setValues({ ...values, tenMonHoc:
e.target.value })}
            />
            {clientErrors.tenMonHoc && <p style={{ color: '#b91c1c',
margin: 0 }}>{clientErrors.tenMonHoc}</p>}
        </div>

        <div style={{ marginBottom: 8 }}>

```

```

        <label>So tin chi</label><br />
        <input
            type="number"
            value={values.soTinChi}
            onChange={(e) => setValues({ ...values, soTinChi:
e.target.value })}
        />
        {clientErrors.soTinChi && <p style={{ color: '#b91c1c',
margin: 0 }}>{clientErrors.soTinChi}</p>}
    </div>

    <div style={{ marginBottom: 8 }}>
        <label>So cho toi da</label><br />
        <input
            type="number"
            value={values.soChoToiDa}
            onChange={(e) => setValues({ ...values, soChoToiDa:
e.target.value })}
        />
        {clientErrors.soChoToiDa && <p style={{ color: '#b91c1c',
margin: 0 }}>{clientErrors.soChoToiDa}</p>}
    </div>

    {serverError && <p style={{ color: '#b91c1c'
}}>{serverError}</p>}

    <button type="submit" disabled={submitting}>
        {submitting ? 'Dang luu...' : (editingCourse ? 'Cap nhat' :
'Them moi')}
    </button>
    {editingCourse && (
        <button type="button" onClick={onCancel} style={{ marginLeft:
8 }}>
            Huy
        </button>
    )}
</form>
);
}

```

Vì sao có cả validate Client lẫn validate Server (đã có từ Buổi 2)? Validate Client giúp phản hồi tức thì, đỡ tốn 1 lượt gọi API cho lỗi hiển nhiên (ví dụ để trống tên môn học). Validate Server (qua Gateway tới course-service, dùng @Valid + CourseDTO) là lớp bảo vệ bắt buộc, không được bỏ - vì Client có thể bị qua mặt (gọi thẳng API bằng Postman, chỉnh sửa code JS bằng DevTools). Không bao giờ được tin tưởng tuyệt đối vào validate phía Client.

D. Cập nhật api/courseApi.ts - thêm hàm tạo/sửa/xoá

```
// path: crs-frontend/src/api/courseApi.ts
// purpose: bổ sung createCourse/updateCourse/deleteCourse, giữ nguyên getCourses từ Buổi 5

import axiosClient from '../axiosClient';
import type { Course, PagedResponse, CourseFormValues } from '../types/course';

export const getCourses = (keyword?: string, page = 0, size = 10) => {
  return axiosClient.get<PagedResponse<Course>>('/api/courses', {
    params: { keyword, page, size },
  });
};

const toPayload = (values: CourseFormValues) => ({
  tenMonHoc: values.tenMonHoc.trim(),
  soTinChi: Number(values.soTinChi),
  soChoToiDa: Number(values.soChoToiDa),
});

export const createCourse = (values: CourseFormValues) => {
  return axiosClient.post<Course>('/api/courses', toPayload(values));
};

export const updateCourse = (id: number, values: CourseFormValues) => {
  return axiosClient.put<Course>(`/api/courses/${id}`, toPayload(values));
};

export const deleteCourse = (id: number) => {
  return axiosClient.delete(`/api/courses/${id}`);
};
```

E. Cập nhật CourseList - thêm nút Sửa/Xoá trên mỗi dòng

```
// path: crs-frontend/src/components/CourseList.tsx
// purpose: bổ sung nút Sửa/Xoá trên mỗi dòng, giữ nguyên xử lý 4 trạng thái từ Buổi 6
```

```

import type { Course } from '../types/course';
import type { LoadState } from '../api/useCourses';

interface CourseListProps {
  courses: Course[];
  state: LoadState;
  errorMessage: string;
  onRetry: () => void;
  onEdit: (course: Course) => void;
  onDelete: (course: Course) => void;
}

export default function CourseList({
  courses, state, errorMessage, onRetry, onEdit, onDelete,
}: CourseListProps) {
  if (state === 'loading') return <p>Đang tải danh sách môn
hoc...</p>;

  if (state === 'error') {
    return (
      <div style={{ color: '#b91c1c' }}>
        <p>{errorMessage}</p>
        <button onClick={onRetry}>Thử lại</button>
      </div>
    );
  }

  if (state === 'empty') return <p>Không tìm thấy môn học nào phù
hop.</p>;

  return (
    <table style={{ width: '100%', borderCollapse: 'collapse' }}>
      <thead>
        <tr style={{ textAlign: 'left', borderBottom: '2px solid #333' }}>
          <th>Tên môn học</th>
          <th>Số tín chỉ</th>
          <th>Số cho còn lại</th>
          <th>Thao tác</th>
        </tr>

```

```

    </thead>
    <tbody>
      {courses.map((course) => (
        <tr key={course.id} style={{ borderBottom: '1px solid #eee'
}}}>
        <td>{course.tenMonHoc}</td>
        <td>{course.soTinChi}</td>
        <td style={{ color: course.soChoConLai === 0 ? '#b91c1c' :
'inherit' }}>
          {course.soChoConLai} / {course.soChoToiDa}
        </td>
        <td>
          <button onClick={() => onEdit(course)}>Sua</button>
          <button onClick={() => onDelete(course)} style={{
marginLeft: 8, color: '#b91c1c' }}>
            Xoa
          </button>
        </td>
      </tr>
    )})}
    </tbody>
  </table>
);
}

```

F. Ráp toàn bộ vào App.tsx

```

// path: crs-frontend/src/App.tsx
// purpose: rap CourseForm + CourseList + Pagination + SearchBox, xu
ly Them/Sua/Xoa
// va dong bo lai danh sach (refetch) sau moi thao tac thanh cong

import { useState } from 'react';
import axios from 'axios';
import { useCourses } from './api/useCourses';
import { createCourse, updateCourse, deleteCourse } from
'./api/courseApi';
import SearchBox from './components/SearchBox';
import CourseList from './components/CourseList';

```

```

import Pagination from './components/Pagination';
import CourseForm from './components/CourseForm';
import type { Course, CourseFormValues } from './types/course';
import type { ApiErrorResponse } from './types/apiError';

function App() {
  const [keyword, setKeyword] = useState('');
  const [page, setPage] = useState(0);
  const [editingCourse, setEditingCourse] = useState<Course | null>(null);
  const [submitting, setSubmitting] = useState(false);
  const [formError, setFormError] = useState<string | null>(null);

  const { courses, totalPages, state, errorMessage, refetch } =
    useCourses(keyword, page);

  const handleSearch = (newKeyword: string) => {
    setKeyword(newKeyword);
    setPage(0);
  };

  const extractErrorMessage = (err: unknown): string => {
    if (axios.isAxiosError<ApiErrorResponse>(err)) {
      const data = err.response?.data;
      if (data?.message) return data.message;
      // Truong hop loi validation server tra ve dang { tenMonHoc:
      "...", soTinChi: "..."}
      if (data) {
        const firstFieldError = Object.values(data).find((v) => typeof
v === 'string');
        if (firstFieldError) return firstFieldError;
      }
    }
    return 'Da xay ra loi, vui long thu lai.';
  };

  const handleFormSubmit = async (values: CourseFormValues) => {
    setSubmitting(true);
    setFormError(null);
    try {

```

```

    if (editingCourse) {
      await updateCourse(editingCourse.id, values);
    } else {
      await createCourse(values);
    }
    setEditingCourse(null);
    refetch(); // dong bo lai danh sach ngay sau khi luu thanh cong
  } catch (err) {
    setFormError(extractErrorMessage(err));
  } finally {
    setSubmitting(false);
  }
};

const handleDelete = async (course: Course) => {
  if (!window.confirm(`Xoa mon hoc "${course.tenMonHoc}"?`)) return;
  try {
    await deleteCourse(course.id);
    refetch();
  } catch (err) {
    alert(extractErrorMessage(err));
  }
};

return (
  <div style={{ padding: 24, fontFamily: 'sans-serif', maxWidth: 800, margin: '0 auto' }}>
    <h1>Quan ly mon hoc (Admin)</h1>

    <CourseForm
      editingCourse={editingCourse}
      onSubmit={handleFormSubmit}
      onCancel={() => setEditingCourse(null)}
      submitting={submitting}
      serverError={formError}
    />

    <SearchBox onSearch={handleSearch} />
    <div style={{ marginTop: 16 }}>
      <CourseList

```

```

        courses={courses}
        state={state}
        errorMessage={errorMessage}
        onRetry={refetch}
        onEdit={setEditingCourse}
        onDelete={handleDelete}
      />
    </div>
    <Pagination currentPage={page} totalPages={totalPages}
onPageChange={setPage} />
  </div>
);
}

export default App;

```

Giải thích extractErrorMessage: hàm này đọc đúng 2 dạng lỗi đã thống nhất từ Buổi 2–3: dạng {"message": "..."} (lỗi nghiệp vụ, ví dụ trùng tên môn học) và dạng {"tenMonHoc": "lỗi", "soTinChi": "lỗi"} (lỗi validation từng field). Đây là ví dụ cụ thể cho thấy lợi ích của việc thống nhất format lỗi xuyên suốt 2 service từ rất sớm.

G. Test theo kịch bản CRUD đầy đủ

Trước khi test, đảm bảo đã set crs_token bằng token của tài khoản admin theo hướng dẫn ở mục A.

#	Thao tác	Kỳ vọng
1	Bấm "Thêm mới" mà không nhập gì, bấm "Them moi"	Hiện đủ 3 lỗi validate Client, KHÔNG gọi API
2	Nhập đầy đủ hợp lệ, bấm "Them moi"	Form đóng lại (về trạng thái Thêm rỗng), danh sách tự cập nhật có môn mới
3	Nhập trùng tên môn học đã có, bấm "Them moi"	Vượt qua validate Client, nhưng server trả lỗi "Ten mon hoc da ton tai", hiện đúng dưới form
4	Bấm "Sửa" trên 1 dòng, form tự động điền sẵn dữ liệu cũ	Đúng dữ liệu hiện tại của dòng đó
5	Sửa tên, bấm "Cập nhật"	Danh sách cập nhật đúng tên mới, form đóng lại
6	Bấm "Sửa" dòng A, sau đó bấm "Sửa" dòng B mà chưa lưu	Form phải đổi sang dữ liệu dòng B, không giữ lại dữ liệu dòng A đang dở
7	Bấm "Xóa", huỷ ở hộp thoại confirm	Không có gì thay đổi
8	Bấm "Xóa", đồng ý ở hộp thoại confirm	Dòng biến mất khỏi danh sách ngay lập tức

